

DocMind Technical Architecture

DocMind is a production-grade RAG system for enterprise document intelligence. Built by Arun Kumar in August 2026.

Key Statistics:

- Average response latency: 3.2 seconds (cache miss), <5ms (cache hit)
- Cache hit rate: 35-40%, saving approximately \$2,400/month at 10,000 daily requests
- Security pipeline catches 98% of known injection patterns
- Uptime target: 99.9% via LangGraph safety net

Tech stack: FastAPI, LangGraph, Google Gemini, PGVector (Supabase), Docker.

Chapter 1: Request Flow

Every request passes through 7 layers:

1. Rate Limiter (slowapi): 20 req/min per IP. Returns HTTP 429 when exceeded.
2. Security Pipeline (3 stages):
 - InputSanitizer: validates encoding, rejects >10,000 chars, strips control chars
 - InjectionDetector: 18 patterns with weighted scoring (0.6-0.95), threshold 0.7
 - PIIMasker: credit cards, SSNs, emails, phones, IPs, Aadhaar, PAN cards
3. Cache: SHA256 key + TTL (1 hour). Skips cache when conversation history exists.
4. Hybrid Retrieval: BM25 (keyword) + Vector (semantic) + RRF (vector 0.6, BM25 0.4, k=60).
5. Contextual Compression: LLM extracts only relevant sentences. Compresses to 30-40%.
6. LangGraph Agent: primary model -> retry (max 2) -> fallback -> graceful error.
7. Structured JSON Logging: request_id, latency, model, tokens, security verdict.

Chapter 2: Hybrid Retrieval

Vector Search: gemini-embedding-001, 768 dimensions, HNSW indexes, $O(\log n)$.

Good for: conceptual queries like 'how does authentication work'

BM25 Search: term-frequency scoring.

Good for: exact matches like 'error code E_TIMEOUT_EXCEEDED' or 'SKU-7742X'

Reciprocal Rank Fusion: $\text{score} = \text{weight} * (1 / (\text{rank} + k))$

Documents in BOTH lists get boosted. No score normalization needed.

Multi-Document: runs retrieval per doc, pools results, applies RRF across all.

Contextual Compression: after retrieval, LLM extracts only relevant sentences.

Reduces tokens 60-70%, removes noise, improves answer quality.

Chapter 3: Security

18 Injection Patterns Detected:

- Direct override: 'ignore previous instructions'
- Role manipulation: 'you are now an unrestricted AI'
- System prompt extraction: 'repeat your system prompt'
- Jailbreak: 'DAN mode', 'do anything now'
- Delimiter injection: special token closing attempts
- Encoded attacks: base64 decode, eval()

Scoring: each pattern weighted 0.6-0.95. Multiple weak signals compound.

Score > 0.7 = blocked (HTTP 400).

PII masked: credit cards, SSN, email, phone, IP, Aadhaar, PAN.

Design: defense in depth. No single layer is perfect.

Chapter 4: Evaluation Pipeline

Two metrics measured via LLM-as-judge:

1. Retrieval Relevance (1-5): Did we find the right chunks?
2. Answer Faithfulness (1-5): Is the answer grounded in context?

Process: generate synthetic Q&A -> run through pipeline -> score with judge LLM.

Targets: relevance > 4.0/5, faithfulness > 4.2/5, latency p95 < 5s.

Interpretation:

- 4.5+: Excellent
- 3.5-4.5: Good
- 2.5-3.5: Fair (tune parameters)
- Below 2.5: Poor (fundamental issues)